

Objects You Already Use

COMSC-1033 Computer Science I

Jeremy Evert

SWOSU

Week 9

Let's begin with values already present in ordinary Python code: a name, a message, or a list of items.

Follow the dot

```
text = "python"
upper_text = text.upper()
print(upper_text)  # expect: PYTHON
print(text)  # expect: python
print(type(upper_text).__name__)  # expect: str
pieces = text.split("h")
print(pieces)  # expect: ['pyt', 'on']
print(type(pieces).__name__)  # expect: list
```

The next clue is a dot.

A string method gives back a value

```
text = "python"
upper_text = text.upper()
print(upper_text)  # expect: PYTHON
print(text)        # expect: python
print(type(upper_text).__name__)  # expect: str
pieces = text.split("h")
print(pieces)      # expect: ['pyt', 'on']
print(type(pieces).__name__)  # expect: list
```

Strings give us a useful surprise. A string method such as uppercase does not edit the original string in place.

A list can change in place

```
route = ["red", "green"]
returned = route.append("blue")
print(route)  # expect: ['red', 'green', 'blue']
print(returned)  # expect: None
backup = route.copy()
print(backup)  # expect: ['red', 'green', 'blue']
print(type(backup).__name__)  # expect: list
```

Now compare a list. Lists are also objects, but a list is a mutable sequence. Mutable means the list can be changed in place.

Use the return value as evidence

```
text = "python"
upper_text = text.upper()
print(upper_text) # expect: PYTHON
print(text) # expect: python
print(type(upper_text).__name__) # expect:
    str
pieces = text.split("h")
print(pieces) # expect: ['pyt', 'on']
print(type(pieces).__name__) # expect: list
```

```
route = ["red", "green"]
returned = route.append("blue")
print(route) # expect: ['red', 'green', '
    blue']
print(returned) # expect: None
backup = route.copy()
print(backup) # expect: ['red', 'green', '
    blue']
print(type(backup).__name__) # expect: list
```

Let's make the contrast concrete. With a string, call the method and print both the returned value and the original.

Here is a small live debugging session. I start with `label = " python "` and write `clean = label.strip()`.

Ask Python what is available

```
signal = "red to green"
print(type(signal).__name__)    # expect: str
print("upper" in dir(signal))   # expect: True
print(signal.replace("green", "blue")) # expect: red to blue
print(signal.find("green"))     # expect: 7
print(signal.startswith("red")) # expect: True
```

You do not need to memorize every method. Python can help you investigate. Type asks what kind of object you have.

Let's collect the evidence before the final review. An object is a value with a type, state, and available behavior.

The review is complete. We met values we already use, followed the dot, and compared what string and list methods do.